list-editor

EECS 280 Project 5: Text Editor

Winter 2026 release.

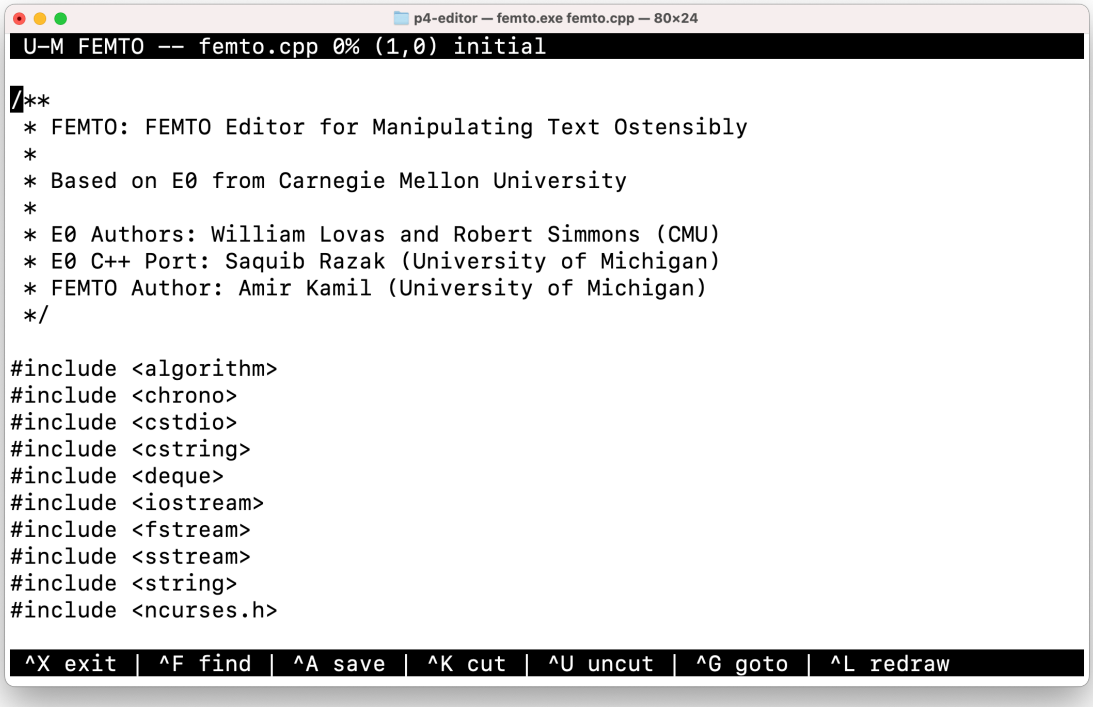
Project due 8:00pm EST Friday, April 10, 2026.

You may work alone or with a partner (partnership guidelines).

Introduction

The learning goals of this project include Container ADTs, Dynamic Memory, The Big Three, Linked Lists, and Iterators. You will gain experience with `new` and `delete`, constructors and destructors, and the `List` data structure that is similar to `std::list` from the standard library.

When you're done, you'll have implemented the basic features of a text editor that is usable through a terminal-based visual editor, similar to other terminal editors such as Pico, nano, Emacs, and vi.



Setup

Set up your visual debugger and version control, then submit to the autograder.

Visual debugger

During setup, name your project `p5-list-editor`. Use this starter files link: `https://eecs280staff.github.io/list-editor/starter-files.tar.gz`

VS Code	Visual Studio	Xcode

You should end up with a folder with starter files that look like this. You may have already renamed files like `List.hpp.starter` to `List.hpp`.

```
$ ls
List.hpp.starter                e0.cpp
List_compile_check.cpp          femto.cpp
List_public_tests.cpp           line.cpp
List_tests.cpp.starter          line_test1.in
Makefile                        line_test1.out.correct
TextBuffer.hpp                  line_test2.in
TextBuffer_public_tests.cpp     line_test2.out.correct
TextBuffer_tests.cpp.starter    unit_test_framework.hpp
```

Here's a short description of each starter file.

File(s)	Description
`List.hpp.starter`	Starter code for the `List` class template.
`List_tests.cpp.starter`	Starter code for your `List` unit tests.
`List_compile_check.cpp`	Compile check test for `List`.
`List_public_tests.cpp`	A very small set of test cases for `List`.
`Makefile`	Helper commands for building.
`TextBuffer.hpp`	Interface specification for the `TextBuffer` class.
`TextBuffer_public_tests.cpp`	Public test cases for the `TextBuffer` class.
`TextBuffer_tests.cpp.starter`	Starter code for your `TextBuffer` unit tests.
`e0.cpp`	A basic visual editor for testing the `TextBuffer` class.
`femto.cpp`	A more functional visual editor that uses the `TextBuffer` class.

File(s)	Description
`line.cpp`	A simple command line tool to test the `TextBuffer` class.

`line_test1.in`
`line_test1.out.correct`
`line_test2.in`
`line_test2.out.correct`
	Input and correct output for system tests with `line.cpp`.

Version control

Set up version control using the Version control tutorial.

After you're done, you should have a local repository with a "clean" status and your local repository should be connected to a remote GitHub repository.

```
$ git status
On branch main
Your branch is up-to-date with 'origin/main'.

nothing to commit, working tree clean
$ git remote -v
origin  https://github.com/awdeorio/p5-list-editor.git (fetch)
origin  https://github.com/awdeorio/p5-list-editor.git (push)
```

You should have a `.gitignore` file (instructions).

```
$ head .gitignore
# This is a sample .gitignore file that's useful for C++ projects.
...
```

Group registration

Register your partnership (or working alone) on the Autograder. Then, submit the code you have.

Linked list

Start by following the instructions in the Setup section below. Then implement your doubly-linked list in `List.hpp`. `List.hpp.starter` provides prototypes for each function. Because `List` is a *templated*, function implementations go in `List.hpp`. There is no `List.cpp`.

While the `List` from lecture was singly linked, this `List` is doubly linked. This `List` also contains an iterator interface. The iterator keeps track of both the current node as well as the `List` to which

it belongs - this allows us to go backwards from an end iterator by looking up the last node in the `List`.

Do not modify the public interface of the `List` class. Implement a doubly-linked list. No arrays or vectors, etc. Manage memory allocation so that there are no memory leaks (Leak checking tutorial).

You can compile and run the provided compile check and `List` tests as follows.

```
$ make List_compile_check.exe
$ make List_public_tests.exe
$ ./List_public_tests.exe
```

Write tests for `List` in `List_tests.cpp` using the Unit Test Framework. You'll submit these tests to the autograder. See the Unit Test Grading section.

```
$ make List_tests.exe
$ ./List_tests.exe
```

> **Pro-tip:** Getting an error about `typename`? Take a look at our reference on Typename.

Setup

Rename these files (VS Code (macOS), VS Code (Windows), Visual Studio, Xcode, CLI):

- `List.hpp.starter` -> `List.hpp`
- `List_tests.cpp.starter` -> `List_tests.cpp`

Edit `List.hpp`, adding a function stub for each member function declared in `List.hpp`, either inside or outside the class definition. Make sure to also add function stubs for the `Iterator` class. Refer to lecture notes for further details. For example:

```
// Inside the class definition:
template <typename T>
class List {
  ...

  bool empty() const {
    assert(false);
  }

  ...
};

```

```
// Or, outside the class definition:
template<typename T>
bool List<T>::empty() const {
  assert(false);
}
```

The List tests should compile and run. The public tests will fail until you implement the functions. The file for your test cases (`List_tests.cpp`) will pass because it initially only contains `ASSERT_TRUE(true)` .

```
$ make List_public_tests.exe
$ ./List_public_tests.exe
$ make List_tests.exe
$ ./List_tests.exe
```

At this point, we haven't written the List Iterator, so `List_compile_check.exe` won't compile. You'll need to take a look at the lecture about iterators and write your own tests. After you do, use the provided compile check like this:

```
$ make List_compile_check.exe
```

Configure your IDE to debug either the public tests or your own tests.

	Public tests	Your own tests
VS Code (macOS)	Set program name to: `${workspaceFolder}/List_public_tests.exe`	Set program name to: `${workspaceFolder}/List_tes`
VS Code (Windows)	Set program name to: `${workspaceFolder}/List_public_tests.exe`	Set program name to: `${workspaceFolder}/List_tes`
Xcode	Include compile sources: `List_public_tests.cpp` , `List.hpp`	Include compile sources: `List_tests.cpp` , `List.hpp`

	Public tests	Your own tests
Visual Studio	Exclude files from the build: • Include `List_public_tests.cpp` • Exclude `List_compile_check.cpp`, `List_tests.cpp`, `TextBuffer_public_tests.cpp`, `TextBuffer_tests.cpp`, `e0.cpp`, `femto.cpp`, `line.cpp`, `main.cpp` (if present)	Exclude files from the build: • Include `List_tests.cpp` • Exclude `List_compile_check.cpp`, `List_public_tests.cpp`, `TextBuffer_public_tests`, `TextBuffer_tests.cpp`, `e`, `femto.cpp`, `line.cpp`, `main.cpp` (if present)

TextBuffer

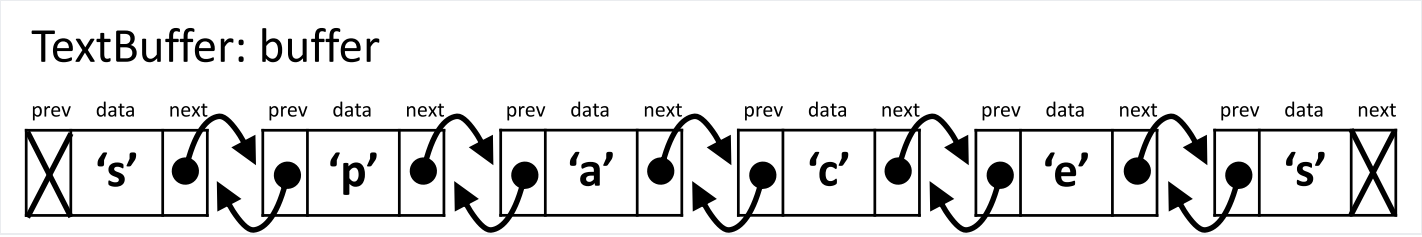
A *text buffer* holds an editable sequence of characters and at the same time has the ability to report the current row and column (You can display this information in Pico/nano with `Ctrl-C`, or in Emacs by typing `M-x line-number-mode` and `M-x column-number-mode`.). The text buffer keeps track of the current cursor position where all edits are done and allows scrolling using the arrow keys to move the cursor through the text.

In this section, you will implement the `TextBuffer` class according the interface defined in the `TextBuffer.hpp` file. The data representation of a `TextBuffer` is a doubly linked list of characters (either your implementation from part 1 or `std::list` from the C++ standard library). The `TextBuffer` also stores an iterator that indicates the current position of the cursor, the current row and column position, and the index of the cursor with respect to the entire buffer (i.e. how many characters from the start to the cursor).

The definition of the `TextBuffer` class is shown below:

```
class TextBuffer {
  using CharList = std::list<char>;
  using Iterator = std::list<char>::iterator;
private:
  CharList data;          // linked list that contains the characters
  Iterator cursor;        // current position within the list
  int row;                // current row
  int column;             // current column
  int index;              // current index
  // ... public interface not shown
};
```

We use `CharList` as a *type alias* for either `List<char>`, or `std::list<char>` (your choice – the autograder will use `std::list<char>` to avoid any issues with your `List` implementation), representing a doubly-linked list of characters. The following list contains the text `spaces`:



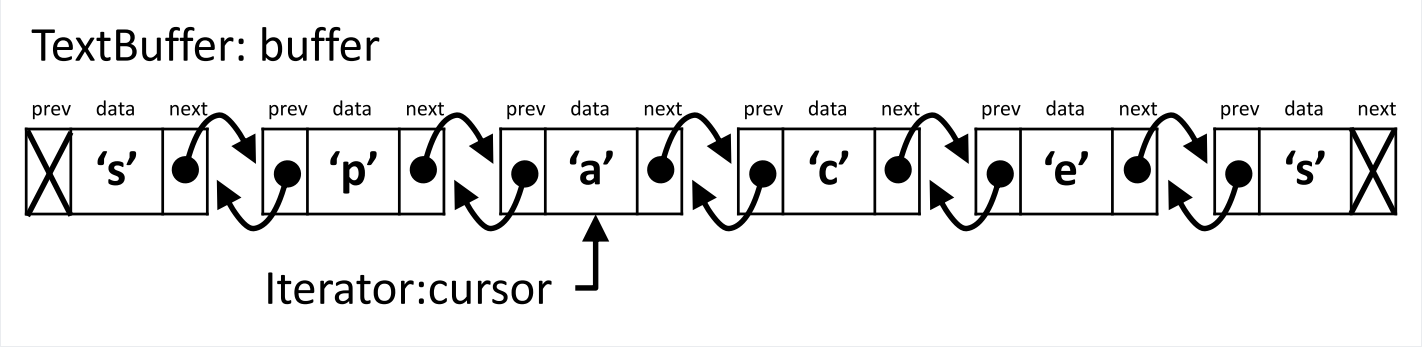
To use a linked list as a text buffer, we need to keep track of the *cursor*, which represents the position where edits can be made in the buffer. In many visual text editors, the cursor appears as a colored rectangle, so that if the cursor was pointing to the node containing the character `'c'`, we'd see it displayed as

```
spaces
```

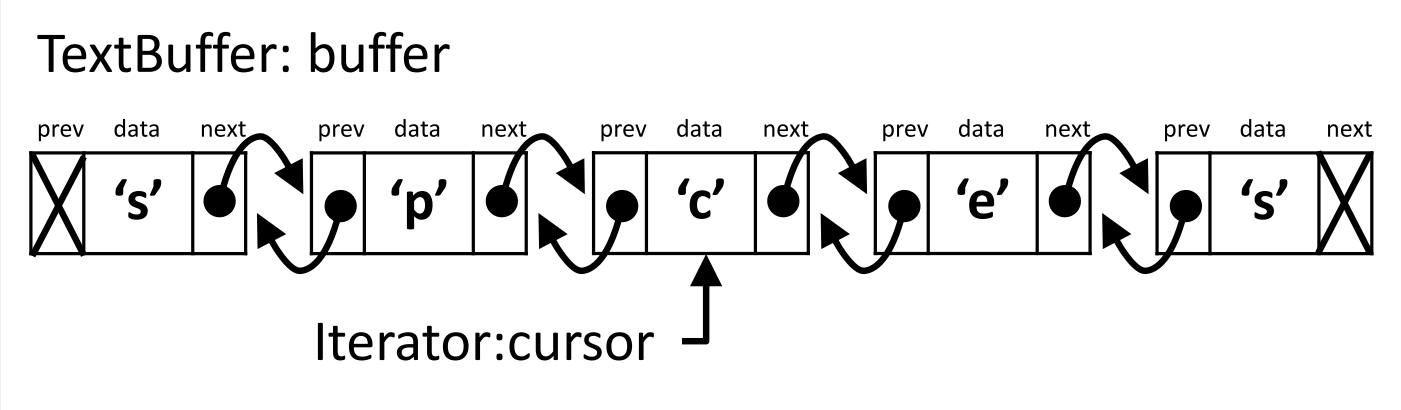
Pressing the left arrow key in a text editor moves the cursor one character backward (to the left).

```
spaces
```

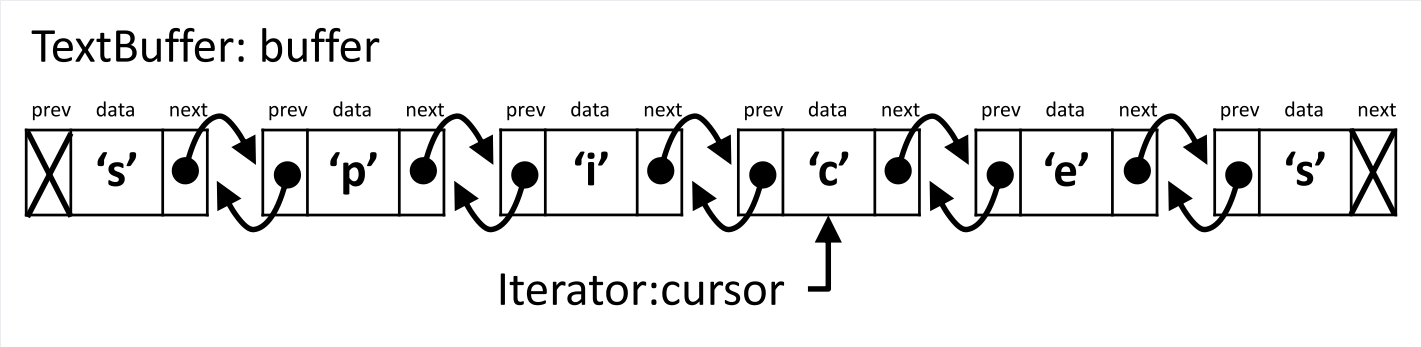
We can now draw the linked list corresponding to this text buffer along with the cursor:



Deletions in a text buffer take place at the cursor. If we delete a character in the previous picture, it will remove the character at the cursor:



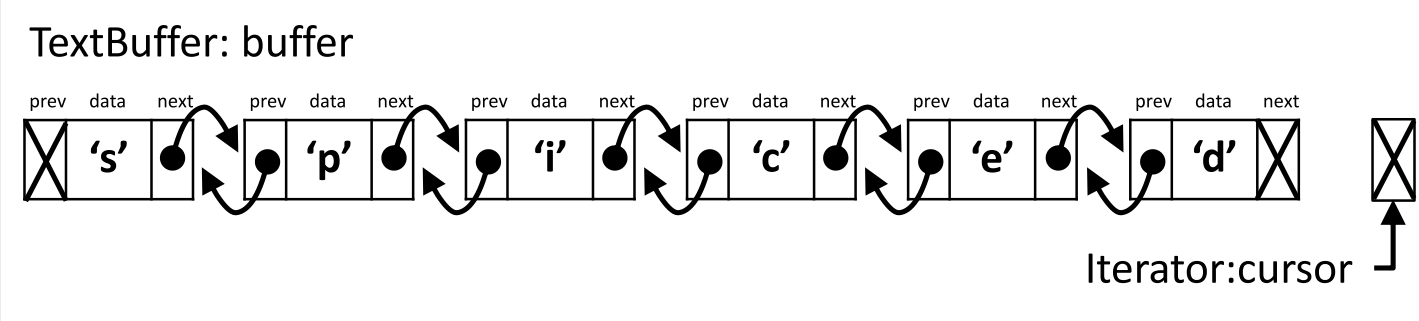
Insertions happen to the left of the cursor. If we next typed the `i` key, that character would be entered in to the left of the cursor.



One consequence of this design is that, in order for additions to be made to the end of the buffer, the cursor needs to be able to go to the right of all the text. In other words, it must be possible for the `cursor` member variable to be equal to the end iterator for the underlying list. Starting from the buffer above, we can see what that looks like from the editor's point of view:

`spices`	(hit the right arrow key to move the cursor forward…)
`spices`	(hit the right arrow key to move the cursor forward…)
`spices`	(hit the delete key…)
`spice`	(type "d"…)
`spiced`	

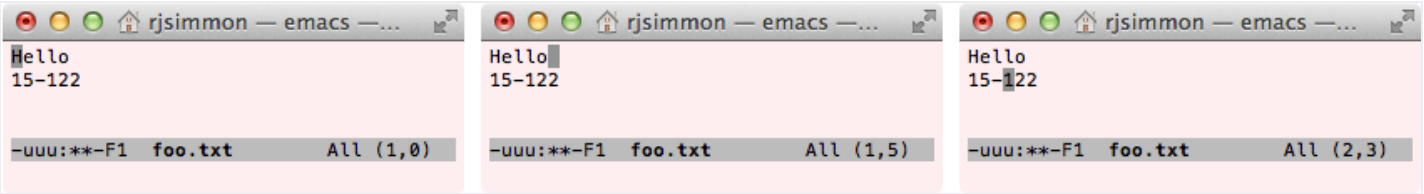
As a doubly-linked list, this final buffer looks like this:



A new, empty text buffer containing no text starts with the cursor at the end iterator.

Rows and Columns

One thing we care a great deal about in a text editor is which characters are newlines, because that is what lets us know our position in the document: the row and column. In Emacs (and the remaining tasks of this assignment), the first row is row 1, and the first column is column 0. In these Emacs buffers, you can see the (row, column) displayed in the lower-right corner:



We can calculate the column of the cursor by working backwards until we find a newline, and we can calculate the row of the cursor by working backwards to the beginning of the buffer and counting the newlines. Note that in the middle example, the cursor is atop a cell containing a newline `\n`, but the cursor is at the end of row 1, not the beginning of row 2.

By tracking the row and column in the data structure, we can report this information to the user without ever having to recalculate the row. It's good to avoid this, because calculating the `row` every time an edit is made can be expensive to run. You should also keep track of the `column` field but sometimes you might need to recalculate this field depending on the type of edit. Any single row is usually relatively short (80 columns maximum, if you're using good style), so this should be fast. Specifically, we only need to recalculate the column when we move left from the beginning of one line to the end of the previous line.

> **Pro-tip:** The `TextBuffer` class has a private `compute_column` member function that is intended to be a helper function for `backward`. Implement `compute_column` before you start implementing `backward`.

In `TextBuffer.cpp`, efficiently implement the interface functions for manipulating editors given in the definition of the `TextBuffer` class in `TextBuffer.hpp`.

Setup

Rename these files (VS Code (macOS), VS Code (Windows), Visual Studio, Xcode, CLI):

- `TextBuffer_tests.cpp.starter` -> `TextBuffer_tests.cpp`

Create a new file `TextBuffer.cpp`. Add function stubs for all the functions in `TextBuffer.hpp`. Remember to `#include "TextBuffer.hpp"`.

Testing the TextBuffer

The `TextBuffer` tests should compile and run once you have created `TextBuffer.cpp` and added function stubs. The public tests will fail until you implement the functions. The file for your test cases (`TextBuffer_tests.cpp`) will pass because it initially only contains `ASSERT_TRUE(true)`.

```
$ make TextBuffer_public_tests.exe
$ ./TextBuffer_public_tests.exe
$ make TextBuffer_tests.exe
$ ./TextBuffer_tests.exe
```

Interactive Testing

You can also test your `TextBuffer` implementation interactively by compiling and running the provided `line.cpp`, which visualizes the contents of a text buffer. Make sure your code passes the tests in `TextBuffer_public_tests.cpp` (see above) before using the interactive tests in `line.cpp`.

```
$ make line.exe
$ ./line.exe
LINE Is Not an Editor -- it is a linear visualization of a TextBuffer.
The '<' character mimics a call to backward()
The '>' character mimics a call to forward()
The '#' character mimics a call to remove()
The '^' character mimics a call to up()
The '!' character mimics a call to down()
The '[' character mimics a call to move_to_row_start()
The ']' character mimics a call to move_to_row_end()
The '@' character mimics a call to insert() with a newline
All other characters just mimic insert() with that character

Give initial input (empty line quits):
```

Try entering `steady^<<<<^>>^>>^@<<@^^` as the initial input. Here are the first few lines of the result:

```
Give initial input (empty line quits):
steady^<<<<^>>^>>^@<<@^^
STARTING
start : |       :(1,0 )
add   : s|      :(1,1 )
add   : st|     :(1,2 )
add   : ste|    :(1,3 )
add   : stea|   :(1,4 )
add   : stead|  :(1,5 )
add   : steady| :(1,6 )
up    : steady| :(1,6 )
left  : stead|y :(1,5 )
left  : stea|dy :(1,4 )
```

The special commands directly invoke the corresponding member functions in the `TextBuffer` class. If an operation cannot be performed (e.g., invoking the `up()` member function to move the cursor up a line when it is already at the first row), the function should leave the text buffer **unchanged** instead of raising an error or assertion violation.

We have provided two test cases that use `line.exe`, which you can run as follows:

```
$ ./line.exe < line_test1.in > line_test1.out
$ diff -qB line_test1.out line_test1.out.correct
$ ./line.exe < line_test2.in > line_test2.out
$ diff -qB line_test2.out line_test2.out.correct
```

Alternatively, use

```
$ make test
```

to run all `List` and `TextBuffer` tests.

Visual Text Editors

The starter files include two visual frontends, `e0` and `femto`, that use your `TextBuffer` class to implement a fully functional, terminal-based editor. You may use these to stress test your `TextBuffer` implementation, but we recommend you ensure your code passes both the `TextBuffer_public_tests` and `line` interactive tests described above before testing with either editor.

⚠ You may need to install the ncurses library before you can use the visual editors. On WSL, run:

```
$ sudo apt install libncurses5-dev
```

On macOS, run:

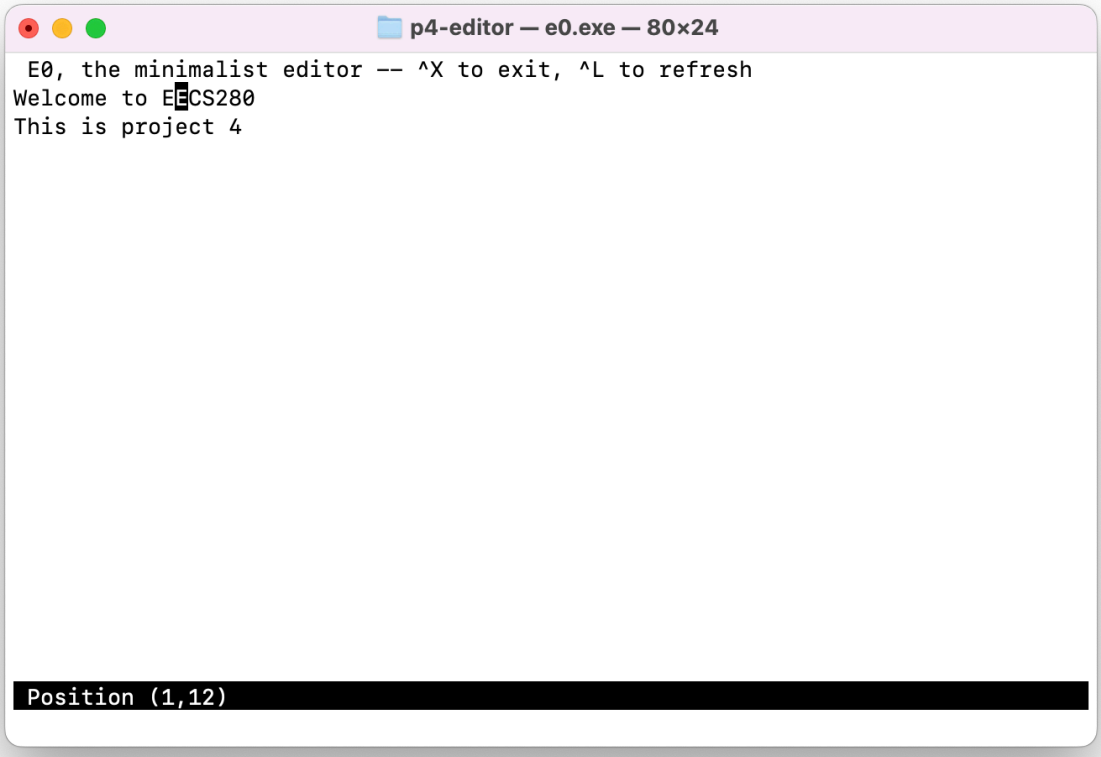
```
$ brew install ncurses
```

Close your terminal and reopen your terminal.

The first visual editor is called *E0*, and you can compile and run it as follows:

```
$ make e0.exe
$ ./e0.exe
```

This will launch a visual editor that will allow you to type text, use delete or backspace to remove characters, and use the arrow and home/end keys to navigate the text. The key combination `Ctrl-X` exits the editor.



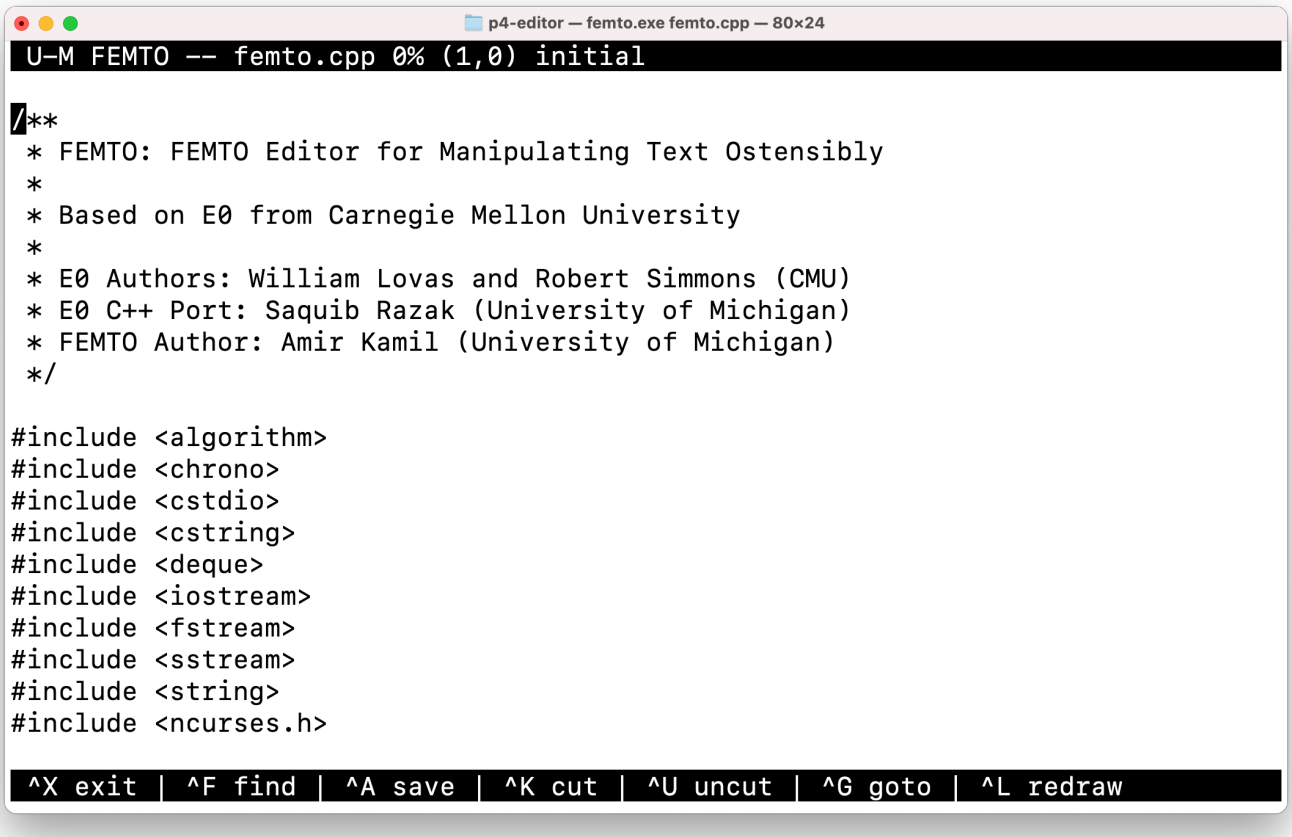
The second visual editor is called *FEMTO*, and it provides much of the functionality of terminal editors such as Pico. Use the following to compile and run it:

```
$ make femto.exe
$ ./femto.exe
```

You can also specify a filename at the command line. For instance, we can examine the visual editor's own source code with

```
$ ./femto.exe femto.cpp
```

You should see something like the following:



You can scroll between pages with the page-up and page-down keys (or just the up and down keys at the top and bottom rows), and you can use the special commands listed at the bottom - for example, `^X` means the combination `Ctrl-X`, which you can use to exit.

Submission and grading

Submit these files to the autograder.

- `List.hpp`

- `List_tests.cpp`
- `TextBuffer.cpp`

This project will be autograded for correctness, comprehensiveness of your test cases, and programming style. See the style checking tutorial for the criteria and how to check your style automatically on CAEN.

Testing

Check for memory leaks using the Leak checking tutorial.

Run all the unit tests and system tests. This includes the public tests we provided and the unit tests that you wrote.

```
$ make test
```

Pro-tip: Run commands in parallel with `make -j`.

```
$ make -j4 test
```

Unit Test Grading

We will autograde your `List` unit tests. We will **not** grade your `TextBuffer` tests.

Your unit tests must use the unit test framework.

A test suite must complete less than 5 seconds and contain 50 or fewer `TEST()` items. One test suite is one `_tests.cpp` file.

To grade your unit tests, we use a set of intentionally buggy instructor solutions. You get points for catching the bugs.

1. We compile and run your unit tests with a **correct solution**.
   - Tests that pass are **valid**.
   - Tests that fail are **invalid**, they falsely report a bug.
2. We compile and run all of your **valid** tests against each **buggy solution**.
   - If any of your tests fail, you caught the bug.
   - You earn points for each bug that you catch.

Requirements and restrictions

It is our goal for you to gain practice with good C++ code, classes, and dynamic memory.

DO	DO NOT
Modify `.cpp` files and `List.hpp`	Modify the signature of any public functions in `List.hpp` or `TextBuffer.hpp`
For `List`, make helper member functions private	Modify the public interface of `List` or `TextBuffer`
Use any part of the STL except for containers in your `List` implementation	Use STL containers in your implementation of `List`
`#include` a library to use its functions	Assume that the compiler will find the library for you (some do, some don't)
Use C++ strings	Use C-strings
Pass large structs or classes by reference	Pass large structs or classes by value
Pass by const reference when appropriate	"I don't think I'll modify it …"
Use the Address Sanitizer to check for memory errors	"It's probably fine…"

Acknowledgments

This project has been adapted from the course 15-122 (Principles of Imperative Computation) offered at Carnegie Mellon University, which is taught in a safe subset of C called *C0*. This project was ported to C++ by Saquib Razak.

This document is licensed under a Creative Commons Attribution-NonCommercial 4.0 License. You're free to copy and share this document, but not to sell it. You may not share source code provided with this document.